

The Legible Node

Tools and patterns for clearer,
more declarative ROS applications



Emerson Knapp

Staff Robotics Engineer

Polymath Robotics



Alistair English

Research Engineer

QUT Centre for Robotics

- **Thin Nodes**

for better ROS code

- **Interface - Working Backward**

from existing systems

- **Interface - Working Forward**

now with 100% more NoDL!

1.

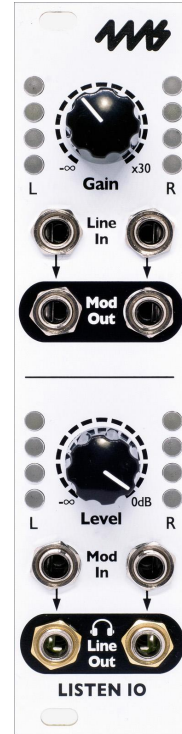
The Node

Building blocks of a ROS application

Nodes

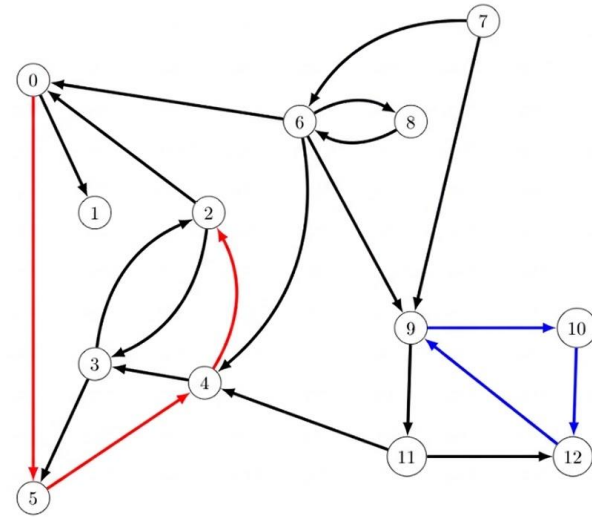
Are the atomic unit of an application.
Connected nodes make a graph, with:

- Settings (Parameters)
- Inputs (Subscriptions)
- Outputs (Publishers)



Applications

A ROS ***application*** is a graph of nodes doing their individual jobs and talking to each other



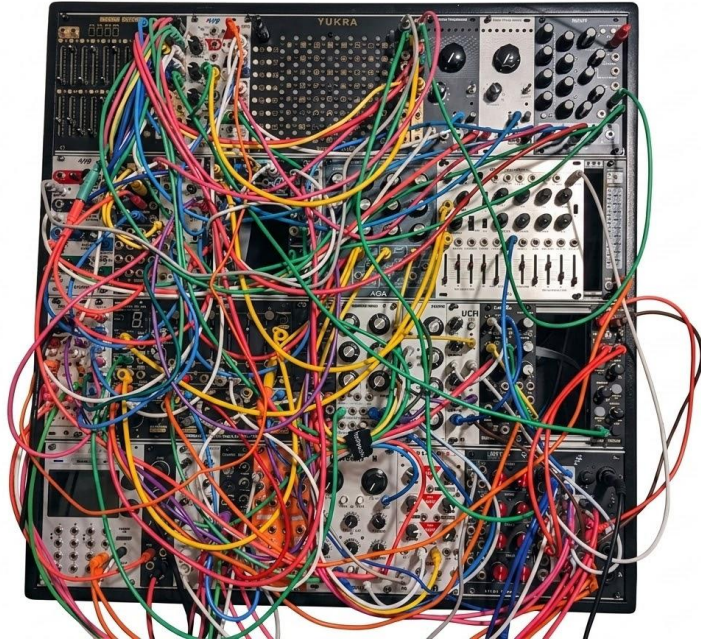
Applications

You might think of it like this...



Applications

But let's be real, it's more like...



Nodes

Writing one is easy!

```
class Talker : public rclcpp::Node
{
public:
    explicit Talker(const rclcpp::NodeOptions & options)
        : Node("talker", options)
    {
        rclcpp::QoS qos(rclcpp::KeepLast{7});
        pub_ = create_publisher<example_interfaces::msg::String>("chatter", qos);
        timer_ = create_wall_timer(1s, std::bind(&Talker::publish_message, this));
    }

    void publish_message() {
        auto msg = std::make_unique<example_interfaces::msg::String>();
        msg->data = "Hello, ROS!";
        pub_->publish(std::move(msg));
    }

private:
    rclcpp::Publisher<example_interfaces::msg::String>::SharedPtr pub_;
    rclcpp::TimerBase::SharedPtr timer_;
};
```

Nodes

Can get complex!



Nodes

Writing them well as they grow...
gets complicated

```
331 this->declare_parameter("dat.set", false);
332 this->declare_parameter("dat.majA", rclcpp::PARAMETER_DOUBLE);
333 this->declare_parameter("dat.flat", rclcpp::PARAMETER_DOUBLE);
334 this->declare_parameter("dat.shift", rclcpp::PARAMETER_DOUBLE_ARRAY);
335 this->declare_parameter("dat.rot", rclcpp::PARAMETER_DOUBLE_ARRAY);
336 this->declare_parameter("dat.scale", rclcpp::PARAMETER_DOUBLE);
337 if (getRosBoolean(this, "dat.set")) {
338     std::vector<double> shift, rot;
339     if (!this->get_parameter("dat.majA", cfg_dat_.maj_a)
340         || !this->get_parameter("dat.flat", cfg_dat_.flat)
341         || !this->get_parameter("dat.shift", shift)
342         || !this->get_parameter("dat.rot", rot)
343         || !this->get_parameter("dat.scale", cfg_dat_.scale)) {
344         throw std::runtime_error(std::string("dat.set is true, therefore ") +
345             "dat.majA, dat.flat, dat.shift, dat.rot, & dat.scale must be set");
346     }
347     if (shift.size() != 3 || rot.size() != 3) {
348         throw std::runtime_error(std::string("size of dat.shift & dat.rot ") +
349             "must be 3");
350     }
351     checkRange(cfg_dat_.maj_a, 6300000.0, 6500000.0, "dat.majA");
352     checkRange(cfg_dat_.flat, 0.0, 500.0, "dat.flat");
353
354     checkRange(shift, 0.0, 500.0, "dat.shift");
355     cfg_dat_.d_x = shift[0];
356     cfg_dat_.d_y = shift[1];
357     cfg_dat_.d_z = shift[2];
358
359     checkRange(rot, -5000.0, 5000.0, "dat.rot");
360     cfg_dat_.rot_x = rot[0];
361     cfg_dat_.rot_y = rot[1];
362     cfg_dat_.rot_z = rot[2];
363
364     checkRange(cfg_dat_.scale, 0.0, 50.0, "scale");
365 }
```

What's the interface?

(hint: you don't know)



Make Interfaces Explicit



REST API endpoints for repositories

Use the REST API to manage repositories on GitHub.

List organization repositories

[Lists repositories for the specified organization](#)

Note

In order to see the `security_and_analysis` block for a repository you must have admin permissions for the repository or be an owner or security manager for the organization that owns the repository. For more information, see ["Managing security managers in your organization"](#).

Fine-grained access tokens for "List organization repositories"

This endpoint works with the following fine-grained token types:

- [GitHub App user access tokens](#)
- [GitHub App installation access tokens](#)
- [Fine-grained personal access tokens](#)

The fine-grained token must have the following permission set:

- `Metadata` repository permissions (read)

This endpoint can be used without authentication or the aforementioned permissions if only public resources are requested.

Parameters for "List organization repositories"

Headers

`Accept` string

Setting to `application/vnd.github.v3+json` is recommended.

Path parameters

`org` string Required

The organization name. The name is not case sensitive.

Code samples for "List organization repositories"

Request example

`curl -X GET https://api.github.com/orgs/octocat/repos`

`curl -X GET https://api.github.com/orgs/octocat/repos`

```
curl -X GET \
  -H "Accept: application/vnd.github.v3+json" \
  -H "Authorization: Bearer <token>" \
  -H "X-GitHub-Api-Version: 2022-05-26" \
  https://api.github.com/orgs/octocat/repos
```

Response

Example response Response schema

Status: 200

```
{
  "type": "list",
  "total_count": 1,
  "items": [
    {
      "name": "HelloWorld",
      "full_name": "octocat/HelloWorld",
      "private": false,
      "visibility": "public",
      "fork": false,
      "source": "https://github.com/octocat/HelloWorld",
      "parent": "https://github.com/octocat/HelloWorld",
      "url": "https://api.github.com/repos/octocat/HelloWorld",
      "html_url": "https://github.com/octocat/HelloWorld",
      "clone_url": "https://github.com/octocat/HelloWorld.git",
      "ssh_url": "https://github.com/octocat/HelloWorld.git",
      "starring_url": "https://api.github.com/repos/octocat/HelloWorld/starring",
      "watching_url": "https://api.github.com/repos/octocat/HelloWorld/watching",
      "subscription_url": "https://api.github.com/repos/octocat/HelloWorld/subscription"
    }
  ]
}
```

Who cares?

Discoverability

A documented interface is usable.
Tools can reason about it.

Testability

Regular old unit tests against
regular old software

Drift Detection

Did your release break the API?
Is the node behaving at runtime?

Static Validation

We can *start* to reason about
launch files (graphs).

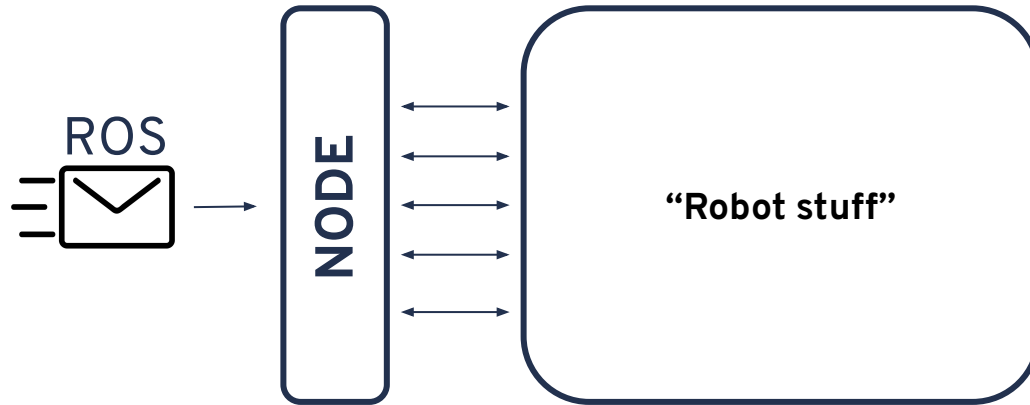
Boilerplate Generation

Less code to write and maintain.

2.

Thin Nodes, Clear Interfaces

A node is an adapter



What belongs in the node?

Configuration

Parameters

Communication

Topics (pub, sub)

Services

Actions

Runtime Model

Timers

Executors

Callback groups

Before

Everything is a callback

```
65 class OccupancyGridMapOutlierFilterComponent : public rclcpp::Node
66 {
67 public:
68     explicit OccupancyGridMapOutlierFilterComponent(const rclcpp::NodeOptions & options);
69
70 private:
71     void onOccupancyGridMapAndPointCloud2(
72         const OccupancyGrid::ConstSharedPtr & input_ogm, const PointCloud2::ConstSharedPtr &
73         input_pc);
74     rclcpp::Publisher<PointCloud2>::SharedPtr pointcloud_pub_;
75     message_filters::Subscriber<OccupancyGrid> occupancy_grid_map_sub_;
76     message_filters::Subscriber<PointCloud2> pointcloud_sub_;
77     std::shared_ptr<message_filters::Synchronizer<SyncPolicy>> sync_ptr_;
78     ...
```

```
288 void OccupancyGridMapOutlierFilterComponent::onOccupancyGridMapAndPointCloud2(
289     const OccupancyGrid::ConstSharedPtr & input_ogm, const PointCloud2::ConstSharedPtr &
290     input_pc)
291 {
292     // Transform to occupancy grid map frame
293     PointCloud2 input_behind_pc{};
294     PointCloud2 input_front_pc{};
295     initializerPointCloud2(*input_pc, input_front_pc);
296     initializerPointCloud2(*input_pc, input_behind_pc);
297     // Split pointcloud into front and behind of the vehicle to reduce the calculation
298     cost
299     splitPointCloudFrontBack(input_pc, input_front_pc, input_behind_pc);
300     finalizePointCloud2(*input_pc, input_front_pc);
```

After

An API you can name

```
namespace outlier_filter
{
    /// @brief Size output to match input's layout and point count, ready to be filled.
    void initializePointCloud2(const PointCloud2 & input, PointCloud2 & output);

    /// @brief Copy input's metadata onto output and recompute its width/row_step from data size.
    void finalizePointCloud2(const PointCloud2 & input, PointCloud2 & output);

    /// @brief Append input's points onto the end of output.
    void concatPointCloud2(PointCloud2 & output, const PointCloud2 & input);

    /// @brief Partition input_pc into points ahead of (front_pc) and behind (behind_pc) the vehicle.
    void splitPointCloudFrontBack(
        const PointCloud2::ConstSharedPtr & input_pc,
        PointCloud2 & front_pc,
        PointCloud2 & behind_pc);

    /// @brief Bucket each point by its occupancy grid cost into high/low confidence and out-of-map clouds.
    void filterByOccupancyGridMap(
        const OccupancyGrid & occupancy_grid_map,
        const PointCloud2 & pointcloud,
        const int cost_threshold,
        PointCloud2 & high_confidence,
        PointCloud2 & low_confidence,
        PointCloud2 & out_ogm);

    /// @brief Full outlier-filter pipeline: split, transform, classify, radius-filter, recombine.
    /// @param radius_search Optional radius filter; when absent, low-confidence points are dropped.
    /// @return Filtered cloud in base_link_frame, or nullptr if a transform fails.
    std::unique_ptr<PointCloud2> filterPipeline(
        const OccupancyGrid::ConstSharedPtr & input_ogm,
        const PointCloud2::ConstSharedPtr & input_pc,
        const std::shared_ptr<tf2_ros::Buffer> tf2,
        std::optional<RadiusSearch2dFilter> radius_search,
        const std::string & base_link_frame,
        const int cost_threshold);
} // namespace outlier_filter
```

Now it's just software

Test

Refactor

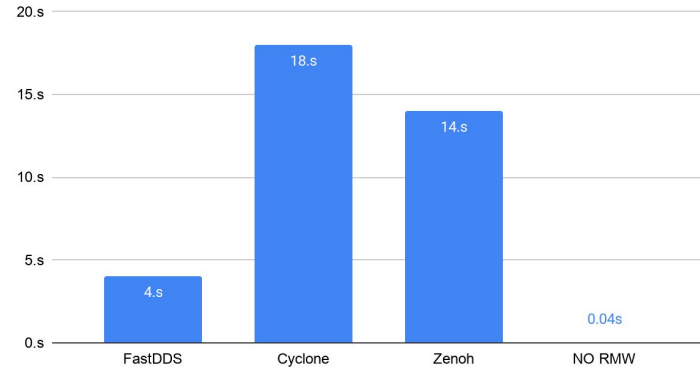
Document

Use it somewhere else

Unit Testing

Overhead from RMW,
not appropriate for logic testing!

--gtest_repeat=1000, default configs

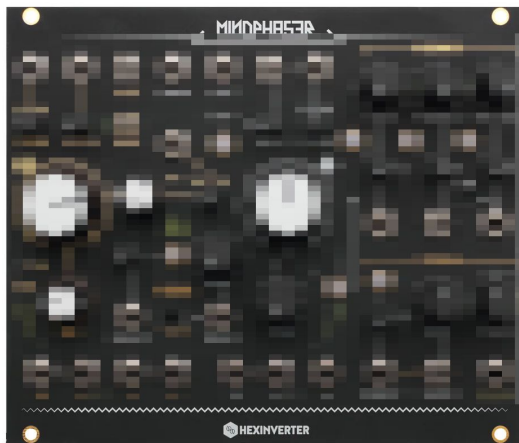


Nodes still have a job

“Thin” is relative, it’s about the separation

But, the ROS interface!

It's still hiding



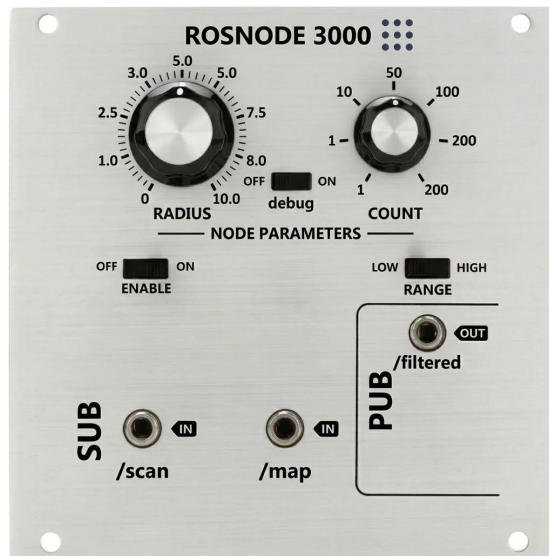
3.

Working Backwards

From nodes to declarations

Node API has no header file

Every \$0.30 circuit component comes with a data sheet



Node APIs

Careful effort required by
individual node maintainers

Inputs / Outputs

Input

Name	Type	Description
~/input/pointcloud	sensor_msgs/PointCloud2	Obstacle point cloud with ground removed.
~/input/occupancy_grid_map	nav_msgs/OccupancyGrid	A map in which the probability of the presence of an obstacle is occupancy probability map

Output

Name	Type	Description
~/output/pointcloud	sensor_msgs/PointCloud2	Point cloud with outliers removed. trajectory
~/output/debug/outlier/pointcloud	sensor_msgs/PointCloud2	Point clouds removed as outliers.
~/output/debug/low_confidence/pointcloud	sensor_msgs/PointCloud2	Point clouds that had a low probability of occupancy in the occupancy grid map. However, it is not considered as an outlier.
~/output/debug/high_confidence/pointcloud	sensor_msgs/PointCloud2	Point clouds that had a high probability of occupancy in the occupancy grid map. trajectory

Parameters

```
{{  
  json_to_markdown("perception/autoware_occupancy_grid_map_outlier_filter/schema/occupancy_grid_map_outlier_filterschema.json")  
}}
```

Node APIs

Even Nav2 has to have handwritten,
paired PRs to update docs

Behavior Server Parameters

`local_costmap_topic`

Type: `string` Default: `"local_costmap/costmap_raw"`

Raw costmap topic for collision checking on the local costmap.

`global_costmap_topic`

Type: `string` Default: `"global_costmap/costmap_raw"`

Raw costmap topic for collision checking on the global costmap.

`local_footprint_topic`

Type: `string` Default: `"local_costmap/published_footprint"`

Topic for footprint in the local costmap frame.

`global_footprint_topic`

Type: `string` Default: `"global_costmap/published_footprint"`

Topic for footprint in the global costmap frame.

Observe the existing system

- `ros2 node info`
- read the source code



or...

Turn topology into data

with `rosgaph_monitor`

`rosgaph_msgs/msg/Node.msg`

```
# Represents the observable runtime state of a ROS Node
# Therefore, does not perfectly align with the abstract specification which created it.

# Fully qualified node name (FQN)
string name

# Parameter specifications for the node
rc_interfaces/ParameterDescriptor[] parameters

# Current values of the node's parameters
# NOTE:
#   parameter_values[] must be empty, or the same size as parameters[]
#   When set, parameter_values[] match 1:1 with the same index in parameters[]
rc_interfaces/ParameterValue[] parameter_values

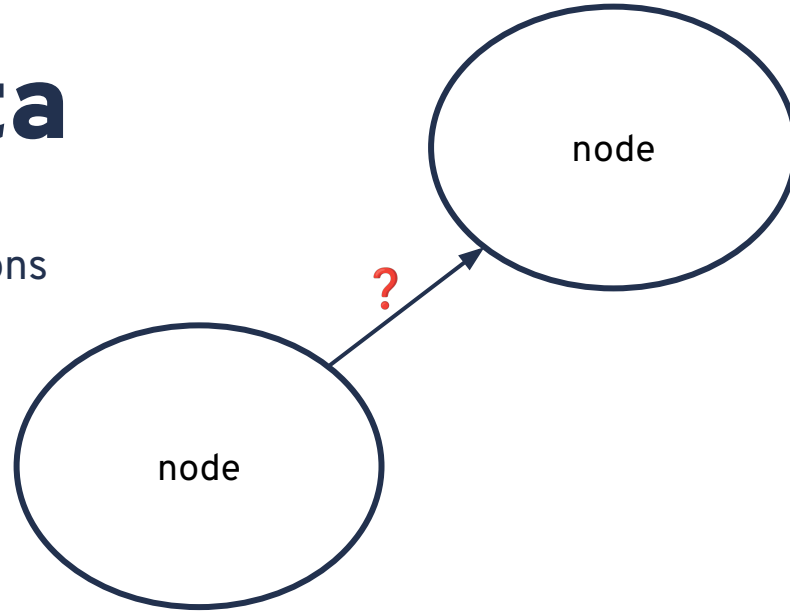
# Communications endpoints - Topics, Services, and Actions
Topic[] publishers
Topic[] subscriptions

Service[] service_clients
Service[] service_servers

Action[] action_clients
Action[] action_servers
```

Once it's data

You can ask (and answer) questions



Observation can recover declarations

At least a best guess starting point



4.

Working Forwards

What if the interfaces existed first?

We already do this for parameters: generate_parameter_library

```
admittance_controller:
  scientific_notation_num:
    type: double
    default_value: 0.000000000001
    description: "Test scientific notation"
    additional_constraints: "Any string can be here."
  interpolation_mode:
    type: string
    default_value: "spline"
    description: "specifies which algorithm to use for interpolation."
    validation:
      one_of<=: [ [ "spline", "linear" ] ]
      "custom_validators::no_args_validator": null
  subset_selection:
    type: string_array
    default_value: ["A", "B"]
    description: "test subset of validator."
    validation:
      subset_of<=: [ [ "A", "B", "C" ] ]
  joints:
    type: string_array
    default_value: ["joint1", "joint2", "joint3"]
    description: "specifies which joints will be used by the controller"
  dof_names:
    type: string_array
    default_value: ["x", "y", "rz"]
    description: "specifies which joints will be used by the controller"
    additional_constraints: "Any string can be here."
```

```
if (param_listener->is_old(params_)) {
  param_listener->refresh_dynamic_parameters();
  params_ = param_listener->get_params();
  RCLCPP_INFO(logger, "Frame: '%s'", params_.control.frame.id.c_str());
  RCLCPP_INFO(logger, "String: '%s'", std::string{params_.fixed_string}.c_str());
}
```

Use generated code

Statically declare your parameters

Introducing: Node Definition Language (NoDL)

```
nodl_version: 2
description: Drives a differential-base robot along a sequence of waypoints.
```

```
include:
- ref: nodl://rclcpp/node
```

```
parameters:
  max_linear_speed:
    type: double
    default_value: 0.8
    description: Upper bound on commanded forward speed, in m/s.
```

```
publishers:
- name: ~/status
  type: std_msgs/msg/String
  qos: {history: KEEP_LAST, depth: 1, reliability: RELIABLE}
  description: Human-readable status of the waypoint follower.
```

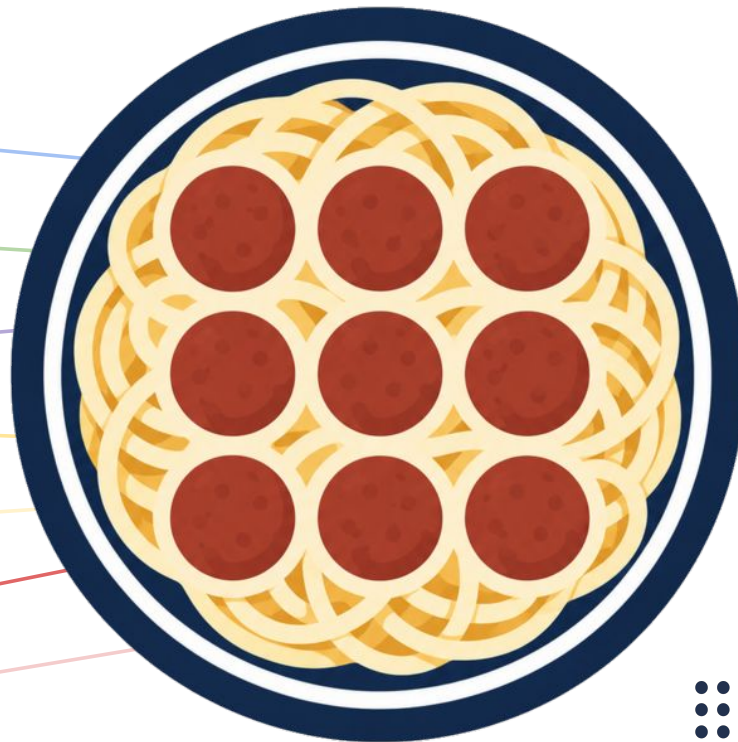
```
subscriptions:
- name: odom
  type: nav_msgs/msg/Odometry
  qos: {history: KEEP_LAST, depth: 10, reliability: BEST_EFFORT}
  description: Wheel/inertial odometry used to track pose.
```

```
service_servers:
- name: ~/cancel
  type: std_srvs/srv/Trigger
  description: Drops any queued waypoints and stops the robot.
```

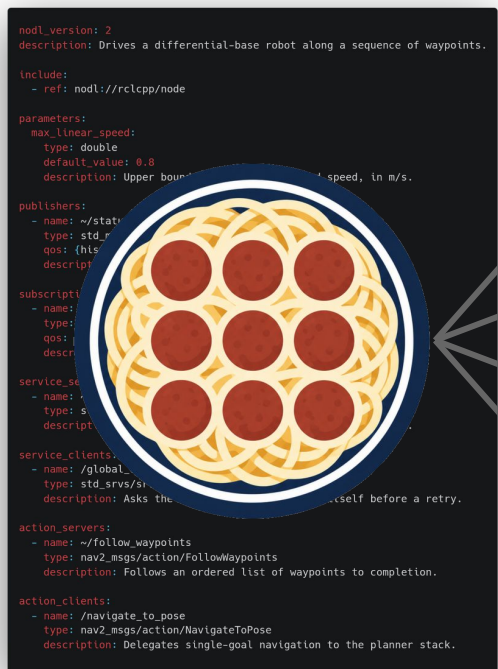
```
service_clients:
- name: /global_costmap/clear_entirely
  type: std_srvs/srv/Trigger
  description: Asks the costmap server to clear itself before a retry.
```

```
action_servers:
- name: ~/follow_waypoints
  type: nav2_msgs/action/FollowWaypoints
  description: Follows an ordered list of waypoints to completion.
```

```
action_clients:
- name: /navigate_to_pose
  type: nav2_msgs/action/NavigateToPose
  description: Delegates single-goal navigation to the planner stack.
```



One declaration, many consumers



Boilerplate Generation

Documentation Generation

Runtime Validation

Graph Analysis

Describe live nodes, generate a NoDL doc

```
$ ros2 run demo_nodes_cpp talker
[INFO] [1788818017.457077006] [talker]: Publishing: 'Hello World: 1'
[INFO] [1788818018.457563424] [talker]: Publishing: 'Hello World: 2'
[INFO] [1788818019.457427683] [talker]: Publishing: 'Hello World: 3'
[INFO] [1788818020.457083541] [talker]: Publishing: 'Hello World: 4'
[INFO] [1788818021.457130001] [talker]: Publishing: 'Hello World: 5'
[INFO] [1788818022.457200140] [talker]: Publishing: 'Hello World: 6'
[INFO] [1788818023.457067792] [talker]: Publishing: 'Hello World: 7'
[INFO] [1788818024.457368167] [talker]: Publishing: 'Hello World: 8'
[INFO] [1788818025.457155204] [talker]: Publishing: 'Hello World: 9'
[INFO] [1788818026.457561291] [talker]: Publishing: 'Hello World: 10'
```

```
$ ros2 nodl describe /talker
nodl_version: 2
publishers:
- name: /chatter
  qos:
    depth: 7
    durability: VOLATILE
    history: KEEP_LAST
    liveliness: AUTOMATIC
    reliability: RELIABLE
  type: example_interfaces/msg/String
```

Assert that a node *conforms*

```
nodl_add_conformance_test(thin_ogmof
  EXECUTABLE ${PROJECT_NAME}_node
  NODL_FILE nodl/node.nodl.yaml
  NODE_NAME thin_ogmof
)
```

Register a CMake test

```
$ colcon test --packages-select thin_ogmof
Starting >>> thin_ogmof
Finished <<< thin_ogmof [4.58s] [ with test failures ]
```

```
Summary: 1 package finished [4.72s]
1 package had test failures: thin_ogmof
```

```
$ colcon test-result --verbose
```

```
...
```

```
AssertionError: NoDL conformance failed for '/thin_ogmof':
[missing] publishers '/thin_ogmof/output/pointcloud': expected type
'PointCloud2' was not observed
[missing] subscriptions '/thin_ogmof/input/pointcloud': expected type
'PointCloud2' was not observed
```

Catch conformance errors at test time

Automatically produce *docs*

```
---
nodl_version: 2
description: Occupancy-grid-map-based pointcloud outlier filter.

include:
  - ref: nodl://nodl_common_interfaces/nodl
  - ref: local://radius_search_2d_filter.nodl.yaml
  - ref: local://tf_listener.nodl.yaml

parameters:
  map_frame:
    type: string
    description: Frame the occupancy grid map is expressed in.
    not_empty:
    base_link_frame:
      type: string
      default_value: base_link
      description: Frame the filtered pointcloud is published in.
      validation:
        not_empty:
      cost_threshold:
        type: int
        default_value: 45
        description: |
          Occupancy cost at or above which a cell is treated as a high-confidence
          obstacle. Occupancy costs range from 0 (free) to 100 (occupied).
      validation:
        bounds: [0, 100]

publishers:
  - name: ~/output/pointcloud
    type: sensor_msgs/msg/PointCloud2
    qos: {history: KEEP_LAST, depth: 5, reliability: RELIABLE}

subscriptions:
  - name: ~/input/occupancy_grid_map
    type: nav_msgs/msg/OccupancyGrid
    qos: {history: KEEP_LAST, depth: 5, reliability: BEST_EFFORT, durability: VOLATILE}
    description: Occupancy grid map used to classify points by cost.

  - name: ~/input/pointcloud
    type: sensor_msgs/msg/PointCloud2
    qos: {history: KEEP_LAST, depth: 5, reliability: BEST_EFFORT, durability: VOLATILE}
    description: Raw input pointcloud to be filtered (SensorDataQoS5).
```



nodl_ogmf 0.52.0 documentation

Search

Standard Documents

Documentation

nodl_ogmf

Index

occupancy_grid_map_outlier_filter

Occupancy grid-map-based pointcloud outlier filter.

The interface includes:

- nodl://nodl_common_interfaces/nodl
- local://radius_search_2d_filter.nodl.yaml
- local://tf_listener.nodl.yaml

Parameters

Name	Type	Default	Description
map_frame	string		Frame the occupancy grid map is expressed in. must not be empty
base_link_frame	string	base_link	must not be empty
cost_threshold	int	45	Occupancy cost at or above which a cell is treated as a high-confidence obstacle. Occupancy costs range from 0 (free) to 100 (occupied). must be within bounds [0, 100.0]
use_simulation_time	bool	false	Use simulation time
use_radius_search_2d_filter	bool	true	When true, low-confidence points are passed through the 2D radius-search filter; when false they are dropped outright.
radius_search_2d_filter.search_radius	double	3.0	Neighbour search radius, in metres. must be greater than 0.0
radius_search_2d_filter.min_points_and_distance_ratio	double	400.0	Ratio used to scale the required neighbour count with distance: the point count threshold is ratio / distance, clamped to [min_points, max_points]. must be greater than 0.0
radius_search_2d_filter.min_points	int	4	Lower clamp on the required neighbour count. must be greater than or equal to 0.0
radius_search_2d_filter.max_points	int	70	Upper clamp on the required neighbour count. must be greater than or equal to 0.0
radius_search_2d_filter.max_filter_points_0s	int	10000	Maximum number of points the filter will process in one call. must be greater than 0.0

Publishers

Name	Type	QoS	Description
~/output/pointcloud	sensor_msgs/msg/PointCloud2	KEEP_LAST(5), RELIABLE	Filtered pointcloud, in base_link_frame.
~/parameter_updates	rcl_interfaces/msg/ParameterUpdate	KEEP_LAST(1), RELIABLE	

Subscriptions

Name	Type	QoS	Description
~/input/occupancy_grid_map	nav_msgs/msg/OccupancyGrid	KEEP_LAST(5), BEST_EFFORT, VOLATILE	Occupancy grid map used to classify points by cost.
~/input/pointcloud	sensor_msgs/msg/PointCloud2	KEEP_LAST(5), BEST_EFFORT, VOLATILE	Raw input pointcloud to be filtered (SensorDataQoS5).
/tf	tf2_msgs/msg/TFMessage	KEEP_LAST(100), RELIABLE, VOLATILE	Dynamic transforms consumed by the transform listener.
/tf_static	tf2_msgs/msg/TFMessage	KEEP_LAST(100), RELIABLE, TRANSIENT_LOCAL	Static transforms consumed by the transform listener.

Service Servers

Name	Type
~/describe_parameters	rcl_interfaces/srv/DescribeParameters
~/get_parameters	rcl_interfaces/srv/GetParameters
~/get_parameter_types	rcl_interfaces/srv/GetParameterTypes
~/list_parameters	rcl_interfaces/srv/ListParameters
~/set_parameters	rcl_interfaces/srv/SetParameters
~/set_parameters_atomically	rcl_interfaces/srv/SetParametersAtomically

Generate C++ boilerplate

```
parameters:
  map_frame:
    type: string
    description: Frame the occupancy grid map is expressed in.

publishers:
- name: ~/output/pointcloud
  type: sensor_msgs/msg/PointCloud2
  qos:
    history: KEEP_LAST
    depth: 5
    reliability: RELIABLE

subscriptions:
- name: ~/input/occupancy_grid_map
  type: nav_msgs/msg/OccupancyGrid
  qos:
    history: KEEP_LAST
    depth: 1
    reliability: RELIABLE
```

Declared NoDL

```
class NodlNode final : public OccupancyGridMapOutlierFilterBase
{
public:
  explicit NodlNode(const rclcpp::NodeOptions & options)
    : OccupancyGridMapOutlierFilterBase(options)
  {
    // no setup needed!

    // access parameters on params_
    int map = params_.map_frame;

    // publishers already created
    pub_output_pointcloud_ -> publish(PointCloud2());
  }

private:
  void on_input_occupancy_grid_map(OccupancyGrid::ConstSharedPtr msg)
  override
  {
    ...
  }
}
```

Override generated base class

Generate Python boilerplate

```
parameters:
  map_frame:
    type: string
    description: Frame the occupancy grid map is expressed in.

publishers:
- name: ~/output/pointcloud
  type: sensor_msgs/msg/PointCloud2
  qos:
    history: KEEP_LAST
    depth: 5
    reliability: RELIABLE

subscriptions:
- name: ~/input/occupancy_grid_map
  type: nav_msgs/msg/OccupancyGrid
  qos:
    history: KEEP_LAST
    depth: 1
    reliability: RELIABLE
```

Declared NoDL

```
class NodlNode(OccupancyGridMapOutlierFilterBase):
    def __init__(self, **kwargs):
        super().__init__(**kwargs)

        # no setup needed!

        # access parameters on params_
        map_frame = self.params_.map_frame

        # publishers already created
        self.pub_output_pointcloud.publish(PointCloud2())

    def on_input_occupancy_grid_map(self, msg):
        # implement callback
        ...
```

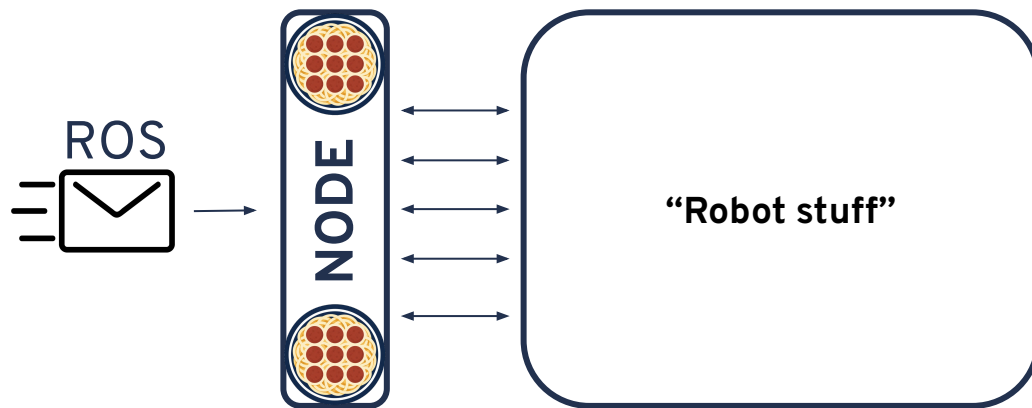
Override generated base class

Generate <your favourite rcl language> boilerplate!

nodl_generator_common	alistair/nodl generator common provenance (#159)	last week
nodl_generator_cpp	alistair/nodl generator common provenance (#159)	last week
nodl_generator_py	Generate rclpy services, actions, and parameters (#162)	yesterday

NoDL is designed to be extensible

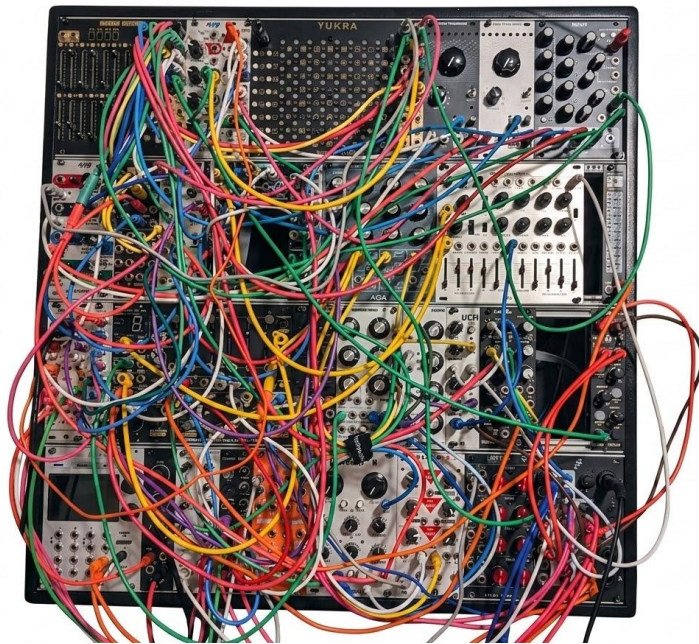
NoDL produces thin nodes

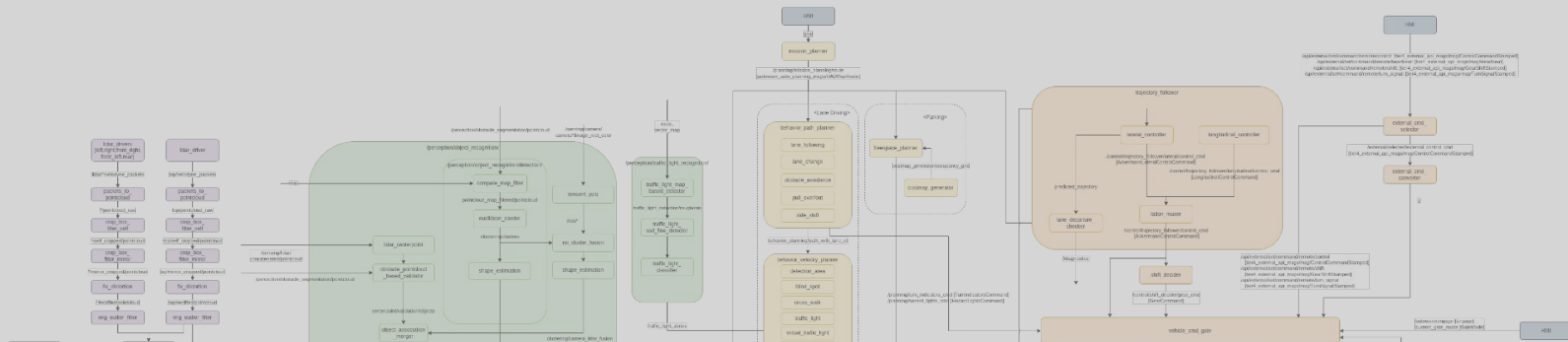


5.

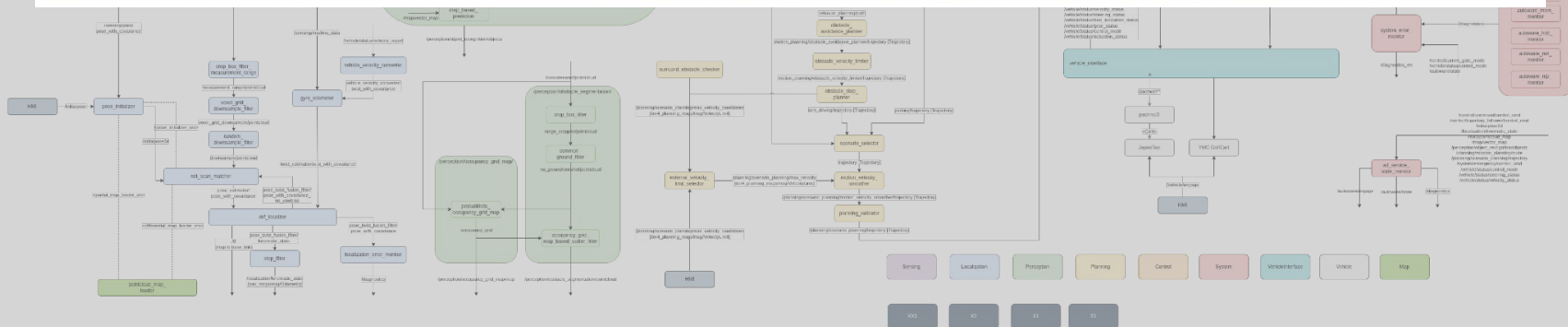
From nodes to systems

Remember this?

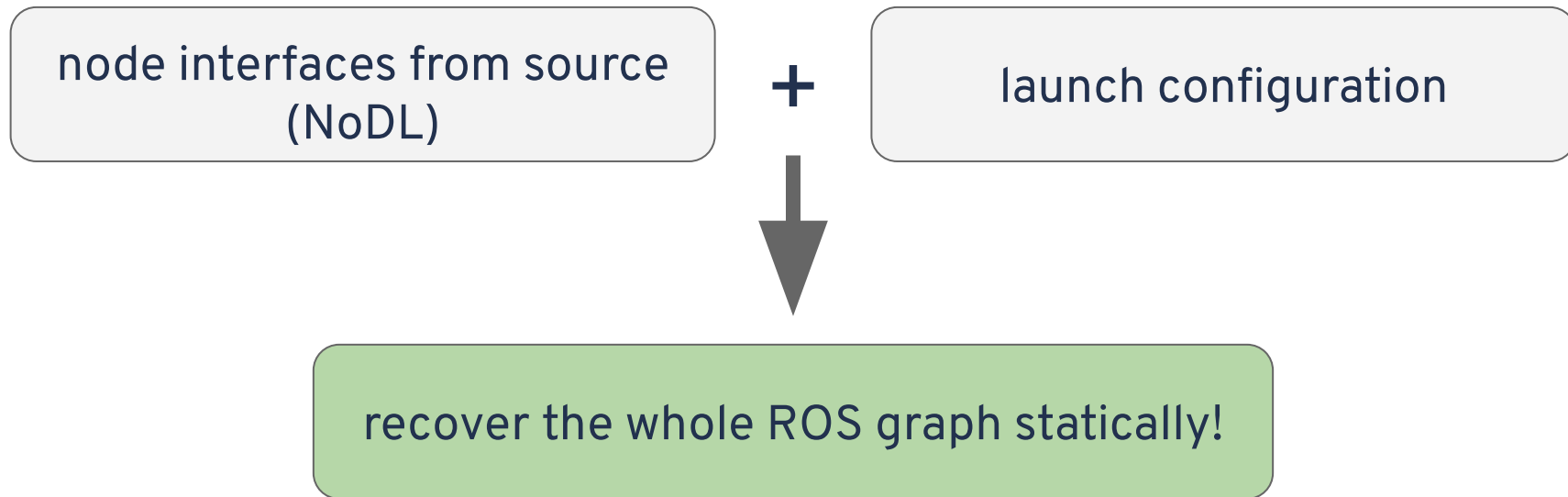




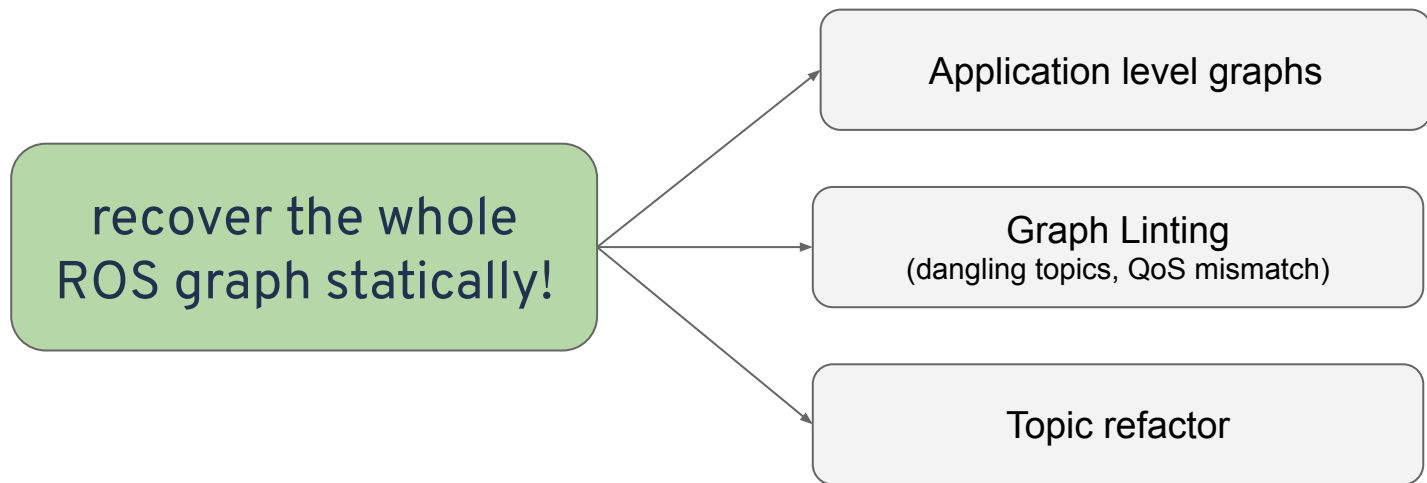
Note that the diagram is for reference. We are planning to update this diagram every release and **may have old information between the releases**. If you wish to check the **latest node diagram** use `rqt_graph` after launching the Autoware.



Looking forward: your ROS graph, statically discovered



Looking forward: your ROS graph, statically discovered



6.

So, what do I do with this?

Homework time!

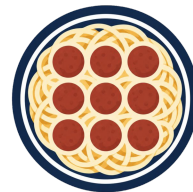


Observe

Work backward
from existing systems

Separate

Thin Nodes
just a ROS adapter



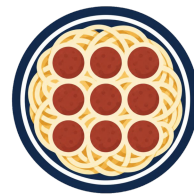
Declare

Work forward
write NoDL, generate



1 Write Thin Nodes
extract logic out of ROS node

2 ros2 nodl describe
existing nodes



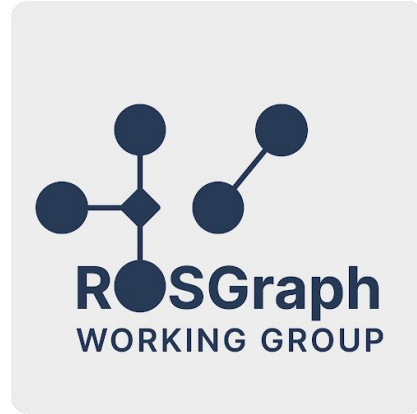
3 Generate docs &
conformance tests

Do this

(then consider code generation)

Join us!

We're just concerned citizens
working for a brighter,
more declarative tomorrow.



Thanks

ROSGraph WG



Luke Sy
@lsy3

PickNik & generate_parameter_library maintainers



Tyler Weaver
@tylerjw

Ubuntu Robotics team



Kyle Fazzari
@kyrofa

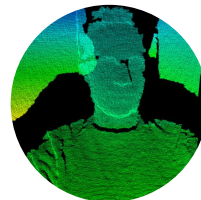


Jeremie Deray
@artivis

Example package original authors



William Woodall
@wjwwood



Julius Kammerl
@jkammerl

Questions?



NoDL

<https://nodl.readthedocs.io>



ROSgraph
Working Group



Sample code &
slides